

Cross-Site Scripting (XSS): real impact and effective mitigation

Tiago Pinto

¹ Universidade de Évora

² Mestrado em Engenharia Informática

³ m68106@alunos.uevora.pt

Abstract. Cross-Site Scripting (XSS) is one of the most common and persistent vulnerabilities affecting modern web applications. By injecting malicious scripts into trusted websites, attackers can compromise user sessions, steal sensitive information, manipulate webpage content, and perform unauthorized actions on behalf of victims.

This paper examines the fundamental concepts behind XSS and analyzes its main variants: Stored XSS, Reflected XSS, and DOM-Based XSS. The study also explores the real-world impact of XSS attacks through practical consequences and documented security incidents. Furthermore, several mitigation techniques are evaluated, including input validation, output encoding, HTML sanitization, Content Security Policy (CSP), secure cookie configurations, and framework-level protections.

The results demonstrate that no single mitigation technique is sufficient on its own. Effective protection against XSS requires a defense-in-depth approach that combines multiple security controls and secure development practices.

Keywords: Cross-Site Scripting · Web Applications · vulnerabilities.

1 Introduction

Web applications have become a fundamental part of modern society, supporting services such as online banking, e-commerce, social media, healthcare, and education. Their accessibility through web browsers and their ability to provide interactive services across different devices have contributed to their widespread adoption [3]. However, as web applications increasingly handle sensitive information and critical business processes, ensuring their security has become a major concern.

Among the various threats targeting web applications, Cross-Site Scripting (XSS) remains one of the most common and dangerous vulnerabilities. XSS occurs when attackers inject malicious scripts into trusted web pages, causing users' browsers to execute unauthorized code. Successful attacks may lead to session hijacking, cookie theft, credential compromise, unauthorized actions, and redirection to malicious websites [2].

Despite being a well-known vulnerability for many years, XSS continues to affect modern web applications due to improper input handling, inadequate output encoding, and the growing complexity of web technologies. As a result, it remains a significant concern for developers, organizations, and security professionals.

This paper examines the real-world impact of Cross-Site Scripting attacks and analyzes the most effective mitigation techniques. It first introduces the fundamental concepts of XSS and its main variants, then explores the consequences of successful attacks, and finally evaluates defensive measures such as input validation, output encoding, Content Security Policy (CSP), and modern framework-based protections.

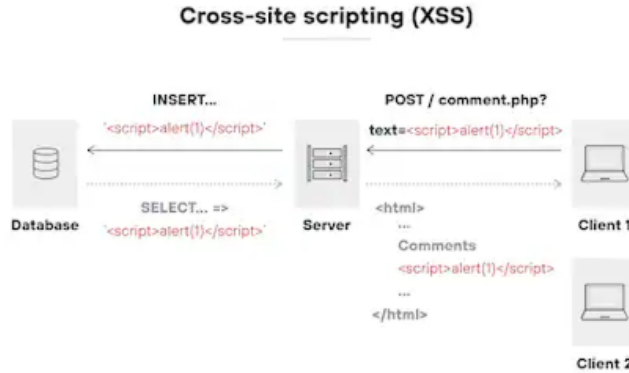


Fig. 1. Stored XSS attack process

2 Background and Fundamental Concepts

To understand Cross-Site Scripting (XSS) vulnerabilities, it is necessary to examine the fundamental technologies that enable modern web applications. XSS exploits the interaction between user input, web application logic, and browser-side script execution. Therefore, understanding how web applications, browsers, and session management mechanisms operate provides the foundation for understanding XSS attacks.

2.1 Web Applications and the Client–Server Model

Web applications are software systems accessed through web browsers over a network, typically the Internet [3]. Unlike traditional desktop applications, they do not require local installation and can be used from different devices and operating systems. Common examples include online banking platforms, e-commerce websites, and social media services.

Most web applications follow the client-server architecture. In this model, the client is the user's web browser, while the server hosts the application logic and data. Communication occurs through HTTP or HTTPS requests and responses. When a user requests a webpage, the browser sends a request to the server, which processes it and returns the required content, such as HTML, CSS, and JavaScript files [4].

Because web applications frequently process user-generated data, improper handling of this input can introduce security vulnerabilities, including XSS.

2.2 Web Browsers, JavaScript, and the DOM

A web browser is responsible for retrieving, interpreting, and displaying web content. Modern browsers not only render webpages but also execute JavaScript code, enabling dynamic and interactive user experiences [5].

When a webpage is loaded, the browser creates a Document Object Model (DOM), a hierarchical representation of the page's structure. JavaScript can interact with the DOM to modify content, update elements, and respond to user actions without requiring a full page reload.

While these capabilities improve usability, they also create security risks. If untrusted user input is inserted into the DOM without proper validation or encoding, malicious JavaScript code may be executed in the user's browser. This behavior forms the basis of many XSS attacks[7].

2.3 Cookies and Session Management

Since HTTP is a stateless protocol, web applications use cookies and session management mechanisms to maintain user authentication and state between requests.

A cookie is a small piece of data stored by the browser on behalf of a website [6]. After a successful login, the server typically creates a session identifier and stores it in a cookie. The browser automatically sends this cookie with subsequent requests, allowing the server to recognize the authenticated user.

Cookies often contain information associated with user sessions, making them valuable targets for attackers. A successful XSS attack may allow malicious scripts to access sensitive information stored in cookies or perform actions on behalf of authenticated users, potentially leading to session hijacking and unauthorized access.

2.4 Relevance to Cross-Site Scripting

Cross-Site Scripting vulnerabilities arise when user-controlled input is processed and returned to the browser without appropriate security controls. Because browsers automatically execute JavaScript contained within trusted webpages, attackers can inject malicious scripts that manipulate the DOM, access sensitive information, or perform unauthorized actions[8].

Consequently, understanding the client-server model, browser behavior, JavaScript execution, the DOM, and session management is essential for understanding how XSS attacks occur and why they remain a significant threat to modern web applications.

3 Types of Cross-Site Scripting Attacks

Cross-Site Scripting (XSS) attacks can be classified into different categories depending on how the malicious script is delivered and executed. Although all XSS attacks involve the execution of attacker-controlled code within a victim's browser, their attack vectors and persistence mechanisms differ significantly. According to OWASP, the three primary types of XSS are Stored XSS, Reflected XSS, and DOM-Based XSS [9].

3.1 Stored Cross-Site Scripting (Stored XSS)

Stored XSS, also known as Persistent XSS, occurs when a malicious payload is permanently stored by a web application and later served to other users. Common targets include comment sections, forums, user profiles, and messaging systems where user-generated content is displayed without proper [10].

When a victim accesses the affected page, the browser retrieves and executes the malicious script as part of the legitimate content. As a result, attackers may steal session information, redirect users to malicious websites, or perform actions on behalf of authenticated users.

A simple example of a malicious payload is:

```
<script>alert("Stored XSS");</script>
```

Stored XSS is generally considered the most severe form of XSS because a single injection can affect multiple users without requiring any additional interaction [8].

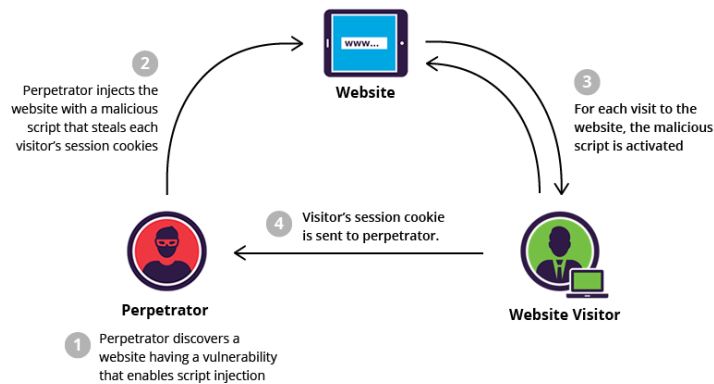


Fig. 2. Stored XSS

3.2 Reflected Cross-Site Scripting (Reflected XSS)

Reflected XSS occurs when user input is immediately returned by the server in an HTTP response without proper validation or encoding. In this scenario, the malicious payload is not stored by the application but is instead delivered through a specially crafted URL [11].

For example:

```
https://example.com/search?q=<script>alert('XSS')</script>
```

If the application reflects the search query directly into the webpage, the browser may execute the injected script. Unlike Stored XSS, Reflected XSS usually requires victims to click a malicious link, making it commonly associated with phishing and social engineering attacks.

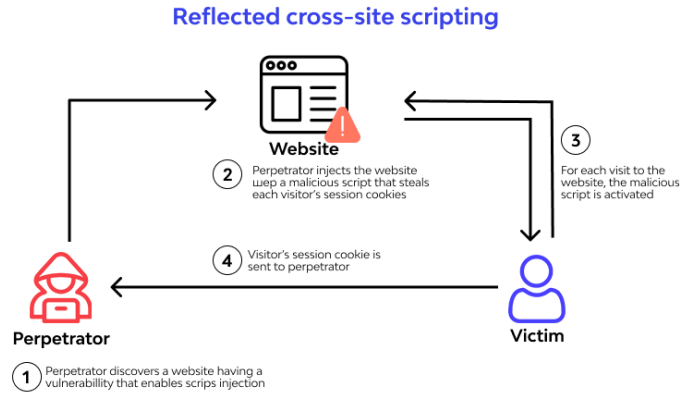


Fig. 3. Reflected XSS

3.3 DOM-Based Cross-Site Scripting (DOM-Based XSS)

DOM-Based XSS occurs entirely on the client side when JavaScript code processes user-controlled data and inserts it into the Document Object Model (DOM) without proper sanitization [7]. In contrast to Stored and Reflected XSS, the vulnerability exists within the browser rather than on the server.

Consider the following example:

```
document.getElementById("output").innerHTML =
location.hash.substring(1);
```

If malicious content is included in the URL fragment, it may be injected into the webpage and executed by the browser. As modern web applications increasingly rely on client-side technologies and dynamic content, DOM-Based XSS has become more common and more difficult to detect.

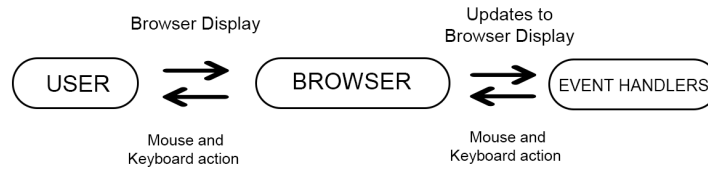


Fig. 4. DOM-Based XSS

3.4 Comparison of XSS Types

Although the objective of all XSS attacks is to execute malicious scripts within a victim's browser, each type differs in its delivery method and execution process.

Type	Payload Storage	User Interaction
Stored XSS	Stored on the server	Not required
Reflected XSS	Not stored	Usually required
DOM-Based XSS	Client-side only	Usually required

Among these categories, Stored XSS is often considered the most dangerous due to its persistent nature and ability to impact multiple users simultaneously. However, all forms of XSS can lead to serious consequences, including session hijacking, credential theft, unauthorized actions, and compromise of sensitive information [12]. Therefore, understanding the characteristics of each attack type is essential for implementing effective mitigation strategies.

4 Real-World Impact of XSS Attacks

Although Cross-Site Scripting (XSS) is often demonstrated using simple JavaScript alert boxes, its real-world impact can be severe. By executing malicious scripts within a trusted web application, attackers can compromise user accounts, steal sensitive information, manipulate website content, and perform unauthorized actions on behalf of victims [2, 8].

4.1 Session Hijacking and Cookie Theft

One of the most common consequences of XSS attacks is session hijacking. Web applications use session identifiers, often stored in cookies, to maintain authenticated user sessions. If an attacker obtains these identifiers through a malicious script, they may impersonate the victim without knowing their credentials [8].

Similarly, XSS can be used to steal browser cookies containing authentication data or user-related information. Although modern security mechanisms such as HttpOnly cookies reduce this risk, improperly configured applications remain vulnerable.

4.2 Credential Theft and Phishing

XSS attacks can also facilitate credential theft through phishing techniques. Attackers may inject fake login forms or modify webpage content to trick users into revealing usernames, passwords, or financial information.

Because the malicious content appears within a legitimate and trusted website, victims are often more likely to provide sensitive information than in traditional phishing attacks [13].

4.3 Website Defacement

Another consequence of XSS is website defacement, where attackers modify the content or appearance of a webpage viewed by users. Through malicious JavaScript, attackers can alter text, images, links, or forms, potentially damaging an organization’s reputation and undermining user trust [14].

Although client-side defacement does not necessarily compromise the server itself, it can significantly affect the perception of the targeted website.

4.4 Keylogging and Token Theft

Malicious scripts injected through XSS vulnerabilities can monitor user activity, including keyboard input. This technique, known as keylogging, may allow attackers to capture usernames, passwords, payment information, and other sensitive data entered into web forms [15].

Modern web applications frequently use JSON Web Tokens (JWTs) and other authentication tokens. If these tokens are stored in browser-accessible locations, attackers may steal them through XSS attacks and use them to gain unauthorized access to protected resources.

4.5 Organizational Impact

Beyond technical consequences, XSS attacks can result in financial losses, reputational damage, regulatory penalties, and loss of customer trust. In many cases, XSS serves as an entry point for more sophisticated attacks by enabling attackers to steal credentials, hijack sessions, or gain access to sensitive information.

Therefore, despite being one of the oldest web vulnerabilities, Cross-Site Scripting continues to represent a significant threat to modern web applications and their users [2].

5 Real-World Case Studies

5.1 Samy Worm (2005)

One of the most famous XSS incidents was the Samy Worm, which targeted the social networking platform MySpace in 2005. The attacker, Samy Kamkar, exploited a Stored XSS vulnerability to create a self-replicating payload that automatically added him as a friend and copied itself to the profiles of infected users [2].

Within approximately twenty hours, the worm had spread to more than one million accounts, making it one of the earliest and most successful examples of a web-based worm. The incident demonstrated how a single Stored XSS vulnerability could rapidly affect a large user population and highlighted the risks associated with user-generated content.

5.2 Twitter OnMouseOver Worm (2010)

In 2010, Twitter experienced a widespread XSS outbreak caused by malicious JavaScript embedded within tweets. The attack exploited browser event handlers and executed whenever users moved their mouse cursor over infected content [13].

Once triggered, the payload automatically spread through the platform, posting messages and redirecting users to external websites. Although Twitter quickly addressed the vulnerability, the incident demonstrated how XSS attacks can propagate rapidly through social media platforms and affect large numbers of users within a short period of time.

5.3 British Airways Data Breach (2018)

In 2018, British Airways suffered a major security incident involving the injection of malicious JavaScript into its online payment pages. The script captured sensitive customer information, including payment card details, and transmitted the data to attacker-controlled servers [16].

Approximately 380,000 transactions were affected during the breach. The incident demonstrated how client-side script injection attacks can compromise sensitive information without disrupting normal website functionality. It also highlighted the financial, legal, and reputational consequences that organizations may face when web application security controls are inadequate.

5.4 Lessons Learned

These incidents demonstrate that Cross-Site Scripting vulnerabilities are not merely theoretical security issues. Whether targeting social networking platforms or commercial services, XSS attacks can spread rapidly, compromise sensitive information, and affect large numbers of users. The continued occurrence of such incidents highlights the importance of implementing effective mitigation techniques and secure development practices throughout the software development lifecycle.

6 Mitigation Techniques and Current Best Practices

Although Cross-Site Scripting (XSS) is one of the oldest web application vulnerabilities, it continues to affect modern systems due to improper handling

of user-supplied data. Preventing XSS requires a combination of secure coding practices, browser security mechanisms, and continuous security testing. No single defense is sufficient; therefore, organizations should adopt a layered security approach [17].

6.1 Input Validation

Input validation is the process of verifying that user-supplied data conforms to predefined rules before being processed by the application. Common validation techniques include restricting data types, limiting input length, and defining acceptable character sets.

A whitelist-based approach, where only expected values are accepted, is generally more secure than blacklisting specific characters or attack patterns. However, input validation alone cannot completely prevent XSS, as legitimate input may still contain characters that can be interpreted as code when displayed in a webpage [8].

6.2 Output Encoding

Output encoding is widely considered the most effective defense against XSS attacks. The objective is to ensure that user input is rendered as text rather than executable code [19].

For example:

```
<script>alert("XSS")</script>
```

can be transformed into:

```
&lt;script&gt;alert("XSS")&lt;/script&gt;
```

As a result, the browser displays the content instead of executing it. Different contexts require different encoding methods, such as HTML, JavaScript, URL, or CSS encoding.

6.3 HTML Sanitization

Some applications, such as forums, blogs, and content management systems, allow users to submit formatted content. In these situations, HTML sanitization can be used to remove potentially dangerous elements while preserving legitimate formatting.

Sanitization tools typically remove:

- `<script>` tags;
- JavaScript event handlers;
- Dangerous attributes;
- Malicious URLs.

Libraries such as DOMPurify are commonly used to sanitize user-generated content before it is displayed to other users [13].

6.4 Content Security Policy (CSP)

Content Security Policy (CSP) is a browser security mechanism that restricts which resources can be loaded and executed within a webpage.

A simple CSP configuration is shown below:

```
Content-Security-Policy:
default-src 'self';
script-src 'self';
```

This policy allows scripts to be executed only if they originate from the application's own domain. Although CSP can significantly reduce the impact of XSS attacks, it should be viewed as an additional layer of protection rather than a complete solution [20].

6.5 Secure Cookie Configuration

Proper cookie configuration can limit the impact of successful XSS attacks.

```
Set-Cookie:
sessionid=abc123;
HttpOnly
```

The **HttpOnly** attribute prevents JavaScript from accessing cookie contents:

Additionally, the **Secure** attribute ensures that cookies are transmitted only through HTTPS connections, while **SameSite** helps protect session data during cross-site requests.

These mechanisms reduce the risk of session hijacking and unauthorized account access [21].

6.6 Secure Development Frameworks

Modern frameworks such as React, Angular, and Vue include built-in protections against XSS. These frameworks automatically escape user-generated content and provide safer methods for rendering dynamic data [17].

However, developers can still introduce vulnerabilities by bypassing security mechanisms or using unsafe functions. Therefore, framework protections should complement, not replace, secure coding practices.

6.7 Security Testing and Defense in Depth

Continuous security testing is essential for identifying vulnerabilities before deployment. Organizations should combine automated scanning tools, code reviews, and penetration testing throughout the software development lifecycle [22].

Since no single mitigation technique is fully effective on its own, modern security strategies rely on a defense-in-depth approach. Combining input validation, output encoding, sanitization, CSP, secure cookie settings, and regular security assessments provides the strongest protection against XSS attacks.

6.8 Summary

Effective XSS mitigation requires multiple complementary security controls. Among the available techniques, output encoding remains the most reliable primary defense, while CSP, secure cookie configurations, and framework-level protections provide additional layers of security. Together, these measures significantly reduce both the likelihood and impact of successful XSS attacks.

7 Critical Evaluation and Discussion

Although Cross-Site Scripting (XSS) is one of the oldest web application vulnerabilities, it remains a significant security issue today. The persistence of XSS demonstrates that security challenges are often related not to the absence of defensive mechanisms, but to the difficulties of implementing them correctly in increasingly complex web applications.

Modern web technologies have improved user experience and application functionality, but they have also expanded the attack surface. The growing use of client-side scripting, third-party libraries, and dynamic content creates more opportunities for developers to introduce security flaws unintentionally.

The real-world incidents discussed in this paper show that XSS is not merely a theoretical vulnerability. Successful attacks can lead to account compromise, data theft, financial losses, and reputational damage, affecting both users and organizations.

Based on the analysis presented, it is evident that preventing XSS requires security to be considered throughout the entire development process. Organizations that adopt secure development practices and regularly assess their applications are better prepared to reduce the risks associated with this vulnerability.

Overall, while significant progress has been made in web application security, XSS continues to represent an important challenge. Its persistence highlights the need for continuous awareness, developer education, and the adoption of secure coding practices in modern web development.

8 Conclusion

Cross-Site Scripting (XSS) remains one of the most prevalent and impactful vulnerabilities affecting modern web applications. Despite decades of research and the availability of numerous defensive mechanisms, XSS continues to pose a significant threat due to the increasing complexity of web technologies and the improper handling of user-supplied data.

This paper explored the fundamental concepts behind XSS, including the technologies that enable web applications and the different forms of XSS attacks, namely Stored, Reflected, and DOM-Based. The analysis demonstrated that successful exploitation can lead to serious consequences such as session hijacking, credential theft, phishing, website defacement, keylogging, and unauthorized access to sensitive information.

Several real-world incidents were examined to highlight the practical impact of XSS vulnerabilities on organizations and users. These cases illustrate that XSS is not merely a theoretical security issue but a persistent threat capable of causing financial losses, reputational damage, and data breaches.

The study also evaluated the most effective mitigation techniques currently available. While output encoding remains the most reliable defense against XSS, additional measures such as input validation, HTML sanitization, Content Security Policy (CSP), secure cookie configurations, and continuous security testing are essential components of a comprehensive security strategy. No single technique is sufficient on its own; therefore, a defense-in-depth approach is required to effectively reduce risk.

In conclusion, protecting modern web applications against XSS requires a combination of secure development practices, multiple defensive layers, and continuous security assessment. As web technologies continue to evolve, organizations must remain proactive in implementing and maintaining effective security controls to mitigate this enduring threat.

References

1. Amazon Web Services (AWS): What is a Web Application? Amazon Web Services. Available at: <https://aws.amazon.com/what-is/web-application/>
2. OWASP Foundation: Cross Site Scripting (XSS). OWASP Community. Available at: <https://owasp.org/www-community/attacks/xss/>
3. Mozilla Developer Network: How the Web Works. Mozilla Foundation. Available at: https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Web_standards/How_the_web_works
4. Mozilla Developer Network: Client-Server Overview. Mozilla Foundation. Available at: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/First_steps/Client-server_overview

5. Mozilla Firefox: What is a Browser? Mozilla Foundation. Available at: <https://www.firefox.com/en-US/more/what-is-a-browser/>
6. Barbera, M.: Understanding Cookies. Google Web.dev. Available at: <https://web.dev/articles/understanding-cookies>
7. Mozilla Developer Network: Cross-site Scripting (XSS). Mozilla Foundation. Available at: <https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/XSS>
8. PortSwigger Ltd.: Cross-site Scripting (XSS). Web Security Academy. Available at: <https://portswigger.net/web-security/cross-site-scripting>
9. OWASP Foundation: Types of Cross-Site Scripting. Available at: https://owasp.org/www-community/Types_of_Cross-Site_Scripting
10. Acunetix: Cross-Site Scripting (XSS). Available at: <https://www.acunetix.com/websitesecurity/xss/>
11. Kaspersky: What is a Cross-Site Scripting Attack? Available at: <https://www.kaspersky.com/resource-center/definitions/what-is-a-cross-site-scripting-attack>
12. Trend Micro: Cross-Site Scripting (XSS) Attacks. Available at: https://www.trendmicro.com/en_us/research/23/e/cross-site-scripting-xss-attacks.html
13. Bright Security: XSS Attack: Types, Examples and Prevention. Available at: <https://brightsec.com/blog/xss-attack/>
14. Imperva: What is a Website Defacement Attack? Available at: <https://www.imperva.com/learn/application-security/website-defacement-attack/>
15. Malwarebytes: What is a Keylogger? Available at: <https://www.malwarebytes.com/keylogger>
16. Ars Technica: British Airways Site Had Credit Card Skimming Code Injected. Ars Technica. Available at: <https://arstechnica.com/information-technology/2018/09/british-airways-site-had-credit-card-skimming-code-injected/>
17. OWASP Foundation: Cross Site Scripting Prevention Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
18. Vaadata: XSS (Cross-Site Scripting) Vulnerabilities: Principles, Types of Attacks, Exploitations and Security Best Practices. Available at: <https://www.vaadata.com/en/>
19. Cloudflare: How to Prevent XSS Attacks. Available at: <https://www.cloudflare.com/learning/security/how-to-prevent-xss-attacks/>
20. Mozilla Developer Network: Content Security Policy (CSP). Mozilla Foundation. Available at: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP>
21. Internet Engineering Task Force: HTTP State Management Mechanism (RFC 6265). Available at: <https://datatracker.ietf.org/doc/html/rfc6265>
22. Rainforest Technologies: Preventing Cross-Site Scripting (XSS) Attacks with Application Security Practices. Available at: <https://www.rainforest.tech/pt-br/appsec/prevenindo-ataques-de-cross-site-scripting-xss-com-praticas-de-appsec/>